

Aprendizaje profundo aplicado a la microestructura de mercados de predicción: predicción de corto plazo en mercados de Bitcoin de Polymarket

Marc Maldonado Lorca
Universidad Politécnica de Madrid
maldonadolorcamar@gmail.com

2026

Resumen

Este trabajo se pregunta si el aprendizaje profundo puede aprovechar las señales de microestructura para anticipar el movimiento de corto plazo del precio de los contratos binarios sobre Bitcoin de Polymarket. A diferencia de la literatura centrada en cómo se resuelve el evento, aquí nos interesan los movimientos locales del precio del contrato en ventanas de segundos; para captarlos combinamos el libro de órdenes con referencias externas del mercado de Bitcoin. La aportación principal no es un modelo concreto, sino una metodología: un sistema reproducible de captura multi-fuente, una validación estrictamente temporal y una evaluación que incorpora costes de ejecución y latencia reales. El hallazgo que vertebra todo lo demás es incómodo. Un clasificador direccional que acierta cerca del 90 % de las veces deja de ser rentable en cuanto se introduce una latencia de entrada de dos segundos: lo que decide el resultado no es acertar la dirección, sino el coste y la ejecución. A partir de ahí recorreremos una progresión de modelos — baselines tabulares, modelos sensibles a la latencia y arquitecturas secuenciales y convolucionales sobre el libro — hasta un especialista tabular congelado que, combinado con un filtro de volatilidad, sobrevive a una validación fuera de muestra de cinco días (+1,069 ticks netos por operación, intervalo de confianza bootstrap del 90 % $[+0,472, +1,654]$, cinco de cinco días positivos). Reportamos cada cifra en ticks netos y siempre junto a su soporte y su incertidumbre. No afirmamos rentabilidad real: lo que se entrega es la metodología, la evidencia obtenida con un protocolo honesto y la delimitación explícita de sus límites.

Palabras clave: microestructura de mercado, mercados de predicción, libro de órdenes, aprendizaje profundo, validación temporal, costes de ejecución, cambio de distribución.

1. Introducción

Polymarket es un mercado de predicción descentralizado donde se negocian contratos binarios sobre eventos futuros mediante un libro de órdenes con emparejamiento fuera de cadena y liquidación en cadena [2]. En los mercados sobre el precio de Bitcoin, el precio del contrato vive entre 0 y 1 y se lee como la probabilidad implícita del evento [1]. Visto así, cada mercado es un pequeño *exchange* con sus bids, sus asks, su spread, su profundidad y su flujo de órdenes. Y eso abre la pregunta que da pie a este trabajo: ¿hay en ese libro de órdenes señales explotables sobre hacia dónde se moverá el precio del contrato en los próximos segundos?

Lo que hace interesante a este entorno es su carácter híbrido. El precio del contrato no es el precio de Bitcoin: es una función no lineal y dependiente del tiempo de la probabilidad de que el evento ocurra. Pero no flota aislado, sino que está acoplado a referencias externas que sí podemos observar —el mercado al contado, los derivados perpetuos y un oráculo de precios. Un modelo útil tiene, entonces, que aprender dos cosas a la vez: la dinámica del libro y la semántica del contrato.

Hay una idea que aparece una y otra vez en este tipo de trabajos: que un clasificador con buena precisión direccional es, por sí solo, una estrategia rentable. Es una idea engañosa. La métrica de clasificación puede ser altísima y, aun así, el sistema no servir para nada si opera con retraso, si aprende artefactos del propio conjunto de datos o si ignora lo que cuesta de verdad cruzar una orden. Por eso aquí cada modelo se juzga no por lo que acierta, sino por lo que gana o pierde una vez se le cargan los costes y la latencia reales.

Contribuciones. Este artículo presenta:

- un sistema de adquisición de datos multi-fuente con controles de calidad por sesión que produce un corpus alineado temporalmente (Sección 3);
- un protocolo de evaluación que combina validación estrictamente temporal, un modelo explícito de costes y latencia medida empíricamente, y la congelación del candidato antes de su validación fuera de muestra (Sección 4);
- evidencia cuantitativa de que la latencia, y no el acierto direccional, determina el resultado, junto con una progresión de modelos que delimita dónde se rompe la cadena entre predicción y ejecución (Sección 5);
- un candidato tabular congelado con filtro de volatilidad que sobrevive a una validación fuera de muestra bajo costes, así como un conjunto de experimentos de adaptación al cambio de régimen con resultado negativo, que reportamos por su valor metodológico (Sección 6).

2. Trabajo relacionado

2.1. Mercados de predicción

Los mercados de predicción se han estudiado sobre todo como mecanismos para agregar información dispersa. Wolfers y Zitzewitz [3] muestran que pueden producir previsiones notablemente precisas, aunque cuánto vale esa previsión depende del diseño del contrato, de la liquidez y de los incentivos de quienes participan. Casi toda esta literatura mira la calibración del precio frente a la resolución final del evento. Nuestro objetivo está un escalón más abajo, en el terreno de la microestructura: la dinámica intradía del precio del contrato a escala de segundos, donde aparecen spreads, profundidad limitada y desajustes pasajeros respecto a las referencias externas.

2.2. Aprendizaje profundo sobre libros de órdenes

Predecir el movimiento del precio a partir del libro de órdenes es un problema con una literatura ya madura. El referente es DeepLOB [4], que combina convoluciones para capturar la estructura espacial del libro con capas recurrentes para la dependencia temporal, y se valida sobre el conjunto de referencia FI-2010 [5]. En la misma línea, Tsantekidis et al. [6] usan redes convolucionales y recurrentes para detectar cambios de precio en el libro. Sirignano y Cont [7] van un paso más allá y muestran que existen características universales en la formación de precios que el aprendizaje profundo es capaz de capturar, y Jha et al. [8] llevan estas ideas al terreno de los cryptoactivos. De aquí tomamos una decisión de diseño: tratar el libro como un objeto con estructura espacio-temporal y no como una tabla plana. Ahora bien, casi todos estos trabajos se evalúan con métricas de clasificación y rara vez con un modelo de ejecución realista, que es justo donde ponemos el acento.

2.3. Microestructura, costes de ejecución y latencia

La distancia entre predecir bien y ganar dinero es uno de los temas clásicos de la microestructura. Kearns y Nevmyvaka [9] repasan el papel del aprendizaje automático en microestructura

y negociación de alta frecuencia, e insisten en algo que aquí se vuelve central: la ejecución, el coste y la latencia deciden si una señal sirve o no. Lo que aportamos es poner número a ese fenómeno en Polymarket. Medimos la latencia real del ciclo de orden y comprobamos que una señal direccional fuerte se vuelve no rentable con una latencia de entrada modesta.

2.4. Aprendizaje automático en finanzas y validación temporal

El aprendizaje automático aplicado a finanzas es terreno fértil para los espejismos en cuanto la evaluación no respeta el orden del tiempo. Gu, Kelly y Xiu [10] dan un marco riguroso para comparar modelos con validación fuera de muestra. López de Prado [11] documenta con detalle cómo se cuela la información del futuro y cómo se sobreajustan los *backtests*, y propone protocolos que respetan la causalidad temporal. Bailey y López de Prado [12] avisan de otro riesgo: si se prueban muchos modelos y se elige el mejor, las métricas de rendimiento salen infladas casi por construcción. Estas tres referencias explican tres decisiones nuestras: la partición estrictamente temporal, el protocolo *out-of-fold* y la congelación del candidato antes de mirar el conjunto fuera de muestra.

2.5. Cambio de distribución y adaptación de dominio

Los mercados no son estacionarios, así que la distribución de los datos cambia entre el periodo en que se entrena el modelo y el momento en que se usa. Para amortiguar ese efecto existen técnicas de adaptación de dominio, como la alineación de correlaciones CORAL [13]. Una de las conclusiones de la Sección 6 es, sin embargo, contraintuitiva: con pocos días de datos, esas técnicas no le ganan a algo mucho más simple, que es restringir la operativa al régimen en el que el modelo aprendió.

3. Adquisición de datos y corpus

3.1. Sistema de captura multi-fuente

Para este trabajo construimos un sistema de captura propio. Por cada sesión de mercado registra el libro de órdenes de Polymarket, el flujo de operaciones, la metadata del mercado y un conjunto de referencias externas: el mercado al contado, los derivados perpetuos de Bitcoin y un oráculo de precios. Todas esas fuentes se alinean en el tiempo sobre una malla regular de dos segundos. La elección de la malla no es un detalle menor, porque fija el contrato de latencia con el que luego evaluamos: en vez de inventar (interpolarse) un instante que nunca observamos, medimos contra la siguiente fotografía real del mercado.

3.2. Controles de calidad por sesión

Antes de entrar en el corpus, cada sesión pasa una serie de controles de calidad: cobertura temporal, continuidad, proporción de datos obsoletos (*stale*), validación del arranque y detección de huecos severos. La que no llega a los umbrales se descarta, sin más. Tirar datos por sistema puede parecer un lujo, pero es una decisión consciente: una referencia externa que llega tarde o incompleta es capaz de fabricar una señal que no existe, y preferimos quedarnos con menos muestra antes que contaminar el entrenamiento con ruido disfrazado de patrón.

3.3. Composición del corpus

El corpus principal abarca del 11 al 25 de mayo de 2026 (entrenamiento, validación y test terminal), con del orden de 2,6 millones de filas cross-venue. Como conjunto fuera de muestra

reservamos del 6 al 10 de junio de 2026, capturado con posterioridad a la congelación del candidato (Sección 4). La unidad de observación es la tupla `sesión × token × instante` sobre la malla de dos segundos.

3.4. Análisis exploratorio del corpus

Antes de entrenar nada, mirar el corpus de cerca deja cuatro lecciones que marcan todo lo que viene después.

La primera tiene que ver con el desequilibrio de clases. La etiqueta direccional está dominada por la clase FLAT: a horizonte H60, entre el 55 y el 65 % de las observaciones caen dentro de la banda de quietud, y el resto se reparte de forma más o menos simétrica entre UP y DOWN. Esto vuelve la precisión bruta profundamente engañosa, porque un modelo que prediga siempre FLAT ya supera el 60 % sin haber aprendido nada. De ahí una de las primeras decisiones del proyecto: no reportar nunca un resultado en *accuracy* a secas, sino apoyarnos en la precisión equilibrada (*balanced accuracy*) y la F1 macro.

La segunda lección es sobre la calidad de los datos. Al revisar la alineación temporal entre las referencias externas y el libro local vimos que, en los tramos de alta volatilidad, los retardos de actualización llegan a abarcar varios ciclos de la malla de dos segundos. Entre el marcador de datos obsoletos y la detección de huecos severos se cae bastante material, y el corpus que queda es bastante más pequeño que el total capturado. Esa escasez es, en buena medida, la razón de la regularización agresiva que aplicamos en todos los modelos.

La tercera apareció al mirar la actividad del libro minuto a minuto: la ventana justo anterior a la apertura del mercado concentra una densidad llamativa de cancelaciones, modificaciones y cruces. Encaja con la idea de que quienes ya están posicionados en el contrato reordenan su exposición antes de que abra, y de paso dejan una señal de microestructura distinta del resto de la sesión. Esa observación es la semilla de la ventana *prestart* y del especialista de la Sección 5.

La cuarta es la que justifica mirar fuera de Polymarket. Al correlacionar el precio del contrato con las referencias de Bitcoin, la correlación es clara en ventanas de minutos pero se diluye en ventanas de segundos. Es decir, el arbitraje externo no es instantáneo. Esa separación de escalas es precisamente lo que hace que merezca la pena combinar el libro local con las referencias externas: en el muy corto plazo, la referencia externa todavía no ha terminado de absorberse en el precio del contrato.

4. Formulación del problema y metodología

4.1. Definición del objetivo

Para cada instante t definimos el retorno futuro del precio medio a un horizonte H ,

$$\Delta_H = \frac{m_{t+H} - m_t}{m_t},$$

donde m_t es el precio medio del contrato. Para pasar de ese retorno continuo a una etiqueta direccional usamos un umbral θ que absorbe el spread, las comisiones y el ruido; todo lo que queda dentro de esa banda se considera quietud,

$$y_t = \begin{cases} \text{UP} & \Delta_H > \theta, \\ \text{DOWN} & \Delta_H < -\theta, \\ \text{FLAT} & \text{en otro caso.} \end{cases}$$

Las características son causales y las etiquetas se calculan *a posteriori*, de forma trazable, así que en ningún momento entra información del futuro por las entradas del modelo. Todos los resultados económicos van en *ticks* netos, que es la unidad natural del libro de órdenes. No los traducimos a dinero a propósito: hacerlo exigiría suponer un tamaño de posición y abriría la puerta a sobrevender los resultados, que es justo lo que queremos evitar.

Objetivos del modelo final. El análisis exploratorio nos empuja a no quedarnos con la etiqueta ternaria para el especialista final, sino a definir dos objetivos complementarios. La etiqueta de tres clases vale para mirar el arco experimental en conjunto, pero un modelo que tiene que decidir si entra o no necesita responder a algo más fino que “¿sube o baja?”: necesita saber cuánto vale la operación y si el entorno en el que va a ejecutarse es favorable.

1. **Regresor de valor esperado** (`exec_net_cost_H60`): neto ejecutado a horizonte H60 descontado el coste de referencia de 0,5 ticks. Un valor predicho positivo indica que el modelo anticipa una operación rentable bajo el modelo de costes; el regresor aprende cuándo hay suficiente movimiento para superar el spread y la comisión.
2. **Clasificador de salud de sesión** (`healthy_fill_proxy_H60`): variable binaria construida como el signo del neto bajo el coste conservador de 0,25 ticks. Actúa como segundo filtro independiente: descarta operaciones en las que el entorno de ejecución es adverso incluso si el valor esperado predicho es positivo. La independencia entre ambas cabezas es deliberada: una operación puede tener alto EV predicho pero baja probabilidad de relleno saludable si el libro está degradado.

Partir el problema en dos objetivos nos deja ser exigentes en dos frentes que no tienen por qué ir juntos: la señal predictiva (cuánto se moverá el precio) y la ejecutabilidad (si el relleno va a ser limpio). Esa doble condición es el corazón de la política congelada que describimos en la Sección 5.

4.2. Validación temporal y prevención de fuga

La partición respeta siempre el orden del tiempo: el entrenamiento se queda con las sesiones más antiguas, la validación con las siguientes y el test terminal con el bloque final. En los experimentos secuenciales añadimos un protocolo *out-of-fold*: cada modelo se entrena solo con días anteriores, las predicciones que recibe la segunda etapa son siempre fuera de muestra, y la política de decisión se fija sin haber mirado el test terminal. Esta disciplina contra la fuga temporal es lo único que nos separa de los resultados ilusorios que la literatura describe una y otra vez [11, 10].

4.3. Modelo de costes y latencia

La latencia no la supusimos: la medimos. En una prueba controlada del ciclo de orden de Polymarket, el tiempo entre ver la oportunidad y que el libro confirme el emparejamiento fue de unos 0,43 segundos en el percentil 95 de la muestra válida. Aun así, por prudencia trabajamos con cifras peores: una latencia conceptual de modelado de un segundo y una latencia observable en los datos de dos segundos. Esta última coincide con la malla de captura, lo que hace la evaluación deliberadamente conservadora. El coste de referencia por operación es de 0,5 ticks, y como contraste optimista usamos 0,25 ticks.

4.4. Protocolo de congelación

El candidato final —el modelo, las 36 características y los umbrales de la política— se congeló antes de ver un solo dato del conjunto del 6 al 10 de junio. Puede sonar a tecnicismo, pero es lo que convierte la validación posterior en una prueba fuera de muestra de verdad y no en otra ronda encubierta de ajuste; es, además, la protección directa contra el sesgo de selección múltiple del que avisan Bailey y López de Prado [12]. Una vez congelada, la política no se vuelve a tocar para “mejorar” los números. Esa renuncia es parte de la tesis.

5. Progresión experimental de modelos

El recorrido por los modelos sigue una regla sencilla: ir de lo simple a lo complejo y, en cada salto, exigir que la complejidad añadida se gane su sitio con evidencia de mejora bajo el mismo protocolo de costes y validación. Si un modelo más sofisticado no supera al anterior con esas reglas, se queda fuera.

5.1. Baseline tabular y el impacto de la latencia

Un baseline tabular sobre el horizonte corto acierta la dirección cerca del 90 % de las veces y pasa la validación temporal por bloques. Mirado solo con métricas de clasificación, parece un modelo excelente. El problema aparece en cuanto se le mete la entrada retrasada y el coste: el panorama se da la vuelta por completo. La Tabla 1 recoge el rendimiento en el test terminal —el bloque que no se usó para elegir la política— según la latencia, en un escenario conservador con filtro de spread.

Latencia	Acciones	Acierto UP	Error DOWN	Neto medio	Neto total
0 s	64	92,19 %	1,56 %	+6,79	+434,44
2 s	63	39,68 %	44,44 %	−1,48	−93,48
4 s	66	45,45 %	43,94 %	−0,74	−48,83
8 s	66	43,94 %	31,82 %	−0,30	−19,53

Tabla 1: Rendimiento en el test terminal bajo distintas latencias (ticks). La señal a latencia nula es real como *markout* desde t , pero no sobrevive como política ejecutable con una latencia de 2 s.

Este es el resultado que vertebra todo el trabajo: acertar la dirección no es lo mismo que ganar. Con una latencia realista de dos segundos, el neto medio del test terminal se hunde hasta −1,48 ticks. El problema, entonces, deja de ser de clasificación y pasa a ser de ejecución, y desde aquí todo modelo se mide por su política económica bajo coste y latencia, no por su acierto.

5.2. Modelos sensibles a la latencia

El paso siguiente fue meter la latencia ya dentro del etiquetado y de la selección. Un modelo sensible a la latencia llegó a un AUC de test de 0,79 —bueno para este tipo de problema— y, pese a ello, su política económica seguía en negativo. La lección refuerza la anterior: tener más capacidad de clasificación no te garantiza una política que gane dinero.

5.3. Modelos secuenciales y convolucionales sobre el libro

Representación secuencial. A partir del corpus construimos dos representaciones compatibles: (i) una secuencia tabular de ocho pasos, en la que cada paso es el vector de características alineadas en ese instante; (ii) un tensor del libro de órdenes de forma $5831 \times 8 \times 10 \times 10$, con los diez primeros niveles del libro en precio y tamaño por cada paso, auditado como completo y alineado sobre la misma malla. La ventana de ocho pasos (16 s en la malla de 2 s) captura suficiente historia local sin incorporar información demasiado lejana que podría no ser relevante a escala de segundos. Los diez primeros niveles eliminan la profundidad vacía del libro sin pérdida de señal relevante.

GRU sobre secuencias tabulares. La primera arquitectura recurrente evaluada es una GRU (*Gated Recurrent Unit*) con capa oculta de dimensión 56 y una única capa recurrente, entrenada con AdamW ($\eta = 0,0015$, decaimiento de pesos 10^{-4}), lotes de 192 ejemplos y paciencia de 18 épocas sin mejora. La elección de GRU frente a LSTM responde a que con aproximadamente

5 800 ejemplos de entrenamiento la diferencia de capacidad entre ambas arquitecturas no compensa el mayor número de parámetros de la LSTM, y la convergencia es más estable. El modelo alcanza métricas de representación razonables (AUC y correlación de rango entre puntuación y delta real), pero las políticas derivadas de su puntuación no son estables entre días.

Codificador convolucional del libro (*BookEncoder*). Para explotar la estructura espacial del libro aplicamos una arquitectura inspirada en DeepLOB [4, 6]. El módulo *BookEncoder* aplica dos capas convolucionales unidimensionales sobre la dimensión de niveles del libro:

$$\text{Conv1d}(c_{\text{in}}, 32, k=3) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0,08) \rightarrow \text{Conv1d}(32, 40, k=3) \rightarrow \text{ReLU},$$

seguido de *mean pooling* y *max pooling* sobre la dimensión temporal, cuyas salidas se concatenan para producir un vector de 80 características. El primer conjunto de convoluciones aprende patrones locales de la estructura del libro (desbalances entre niveles, gradientes de profundidad); el pooling dual captura tanto el nivel promedio como los picos, que en mercados poco líquidos pueden ser más informativos que la media. El número de canales (40) y la tasa de *dropout* (0,08) se fijaron con validación temporal: valores mayores provocaban sobreajuste a las pocas épocas con el horizonte de datos disponible.

Modelo de fusión. Un tercer modelo combina la salida de la GRU (sobre la secuencia tabular) con la del BookEncoder (sobre el tensor del libro) y aplica una cabeza lineal sobre el vector concatenado. En validación cruzada temporal el modelo de fusión mostró la mejor señal de representación individual: la combinación de la secuencia tabular y la estructura del libro aportaba más información que cualquiera por separado. Sin embargo, su política económica resultó positiva únicamente en las métricas *proxy* dentro de muestra. Al imponer selección causal día a día (las predicciones de la segunda etapa deben ser estrictamente fuera de muestra) y exigir estabilidad en todos los días del bloque de test, la ventaja desapareció.

Detección de régimen de volatilidad (TCN). Se evaluó también una red convolucional temporal orientada a clasificar el régimen de volatilidad de la sesión —alto o bajo— según el umbral del percentil 80 de la volatilidad realizada en entrenamiento. La cabeza de régimen de la TCN reprodujo de forma consistente la partición que define el filtro de volatilidad estático. Este experimento confirma dos cosas: que el régimen de volatilidad es la variable que más condiciona la viabilidad de la política, y que un filtro estático simple es suficiente para capturar esa información con los datos disponibles.

Diagnóstico del bloque de aprendizaje profundo. Todos estos experimentos terminan apuntando al mismo sitio. Las arquitecturas profundas aprenden representaciones útiles —del libro y de la secuencia—, pero no consiguen una política estable con apenas quince días de entrenamiento. Con ~5 800 ejemplos, lo que hace falta para generalizar de un día al siguiente queda por encima de lo que estas redes pueden estimar con fiabilidad: aprenden bien los patrones del periodo de entrenamiento, pero no los trasladan a los días que vienen después. Es un comportamiento esperable cuando los datos escasean [10], y es lo que nos hace volver a un modelo tabular con regularización agresiva como candidato final. El aprendizaje profundo no se descarta: se aparca a la espera de un volumen de datos mucho mayor.

5.4. Corrección del score de selección adversa

Conviene contar también un error, porque enseña algo. Durante la auditoría de la selección descubrimos que habíamos interpretado al revés una variable objetivo: en la selección adversa, el valor positivo señala un resultado *desfavorable*, y sin querer estábamos usando su probabilidad como si puntuara hacia abajo. La orientación correcta es $1 - P(\text{adverso})$. Marcamos como

superados todos los resultados afectados y repetimos la validación con el signo bien puesto; la candidata que teníamos hasta entonces dejó de valer como evidencia. Lo dejamos escrito a propósito: una métrica positiva no basta si no se ha auditado qué significa, económicamente, la puntuación que está guiando la selección.

5.5. Protocolo *out-of-fold* y selección de horizonte

Repetimos la fusión, esta vez bajo un protocolo estrictamente fuera de muestra. La segunda etapa no nos dio una política robusta, pero el modelo a horizonte H60 conservó una señal modesta (AUC de test 0,5656; rendimiento medio del 35 % superior tras coste 0,5 de +0,1778 ticks *proxy*). Para descartar que el fallo fuese anticipar demasiado pronto, enfrentamos H60 contra H120 con las mismas secuencias (Tabla 2).

Métrica	H60 emparejado	H120
AUC test	0,5651	0,4951
Top 35 % test, coste 0,5	+0,4262	-1,1171
Top 35 % test, coste 1,0	+0,0579	-1,4910

Tabla 2: Selección de horizonte (ticks). H120 se comporta de forma próxima al azar; H60 es el único que conserva señal. El horizonte H240 colapsa aún más en el conjunto fuera de muestra.

Los horizontes largos quedan descartados, y con datos en la mano: alargarlos no mejora la señal, la empeora. Cerramos ahí el estudio multi-horizonte y H60 se queda como único candidato.

5.6. Especialista de ventana *prestart*

Arquitectura de dos cabezas. El candidato final es un modelo de *gradient boosting* sobre histogramas (HistGradientBoosting de sklearn) con dos cabezas independientes, entrenadas sobre el mismo conjunto de 36 características sin variables de reloj (*no_clock*):

1. **Regresor EV** (objetivo `exec_net_cost_H60`): predice el neto esperado de la operación bajo el coste de referencia de 0,5 ticks.
2. **Clasificador *healthy*** (objetivo `healthy_fill_proxy_H60`): predice la probabilidad de que el relleno sea favorable bajo el coste conservador de 0,25 ticks.

La elección de HistGradientBoosting frente a otras librerías de *gradient boosting* responde a tres razones. Primero, su implementación nativa maneja valores faltantes y variables categóricas sin preprocesamiento explícito, lo que elimina un punto de fuga potencial: una imputación incorrecta podría introducir información que no estaría disponible en producción. Segundo, su interfaz de parada anticipada opera sobre una fracción temporal de la validación interna, lo que es coherente con el protocolo de validación estrictamente temporal. Tercero, la regularización L2 integrada y el control de la complejidad del árbol (número de hojas y muestras mínimas por hoja) permiten expresar el régimen de alta regularización necesario con pocos datos sin hiperparámetros adicionales.

Hiperparámetros y regularización. Ambas cabezas comparten: tasa de aprendizaje 0,045, hasta 220 iteraciones con parada anticipada activa (fracción de validación interna 0,15, paciencia 15 iteraciones sin mejora), máximo de 15 hojas por árbol, mínimo de 120 muestras por hoja y regularización L2 de 0,10. La combinación de árboles con pocas hojas (15), muchas muestras mínimas por hoja (120) y regularización L2 explícita constituye un régimen de regularización agresiva deliberado: con $\sim 1\,000$ observaciones útiles en el bloque de entrenamiento del especialista, es necesario restringir la complejidad para evitar que el modelo aprenda patrones espurios del libro de una sola semana.

Conjunto de características y filtro de selección. El conjunto de 36 características excluye variables con información directa del reloj: horas, minutos y segundos se eliminan para evitar que el modelo aprenda atajos temporales triviales (como “a esta hora del prestart suele haber señal”). Las observaciones se seleccionan con filtros estrictos: ventana *prestart* entre -60 y -45 s respecto a la apertura, libro con cobertura completa $\geq 0,999$, coste visible acotado a 1,25 ticks, edad de la mejor cotización inferior a 1 segundo y ningún indicador de degradación activo.

Política congelada y selección de umbrales. La política opera sobre dos condiciones simultáneas: valor esperado predicho superior a 0,75 y probabilidad de salud no inferior a 0,50. Los umbrales se seleccionaron sobre el conjunto de validación —bloque temporal posterior al de entrenamiento y anterior al de test terminal— entre una malla de valores para el regresor ($-0,25, 0,00, 0,25, 0,50, 0,75, 1,00, 1,50, 2,00$) y para el clasificador ($0,50, 0,55, 0,60$), maximizando el neto por operación con soporte mínimo garantizado. Una vez fijados, los umbrales no se modificaron antes de observar los datos fuera de muestra, respetando el protocolo de congelación descrito en la Sección 4.

5.7. Filtro de volatilidad

Al preparar la validación fuera de muestra nos topamos con un cambio de régimen que no estaba en los planes: la fracción de sesiones de alta volatilidad saltó del 18–22 % del corpus de entrenamiento a alrededor del 58 % en el conjunto fuera de muestra (Tabla 3). El modelo había aprendido en un mundo de baja volatilidad, y sus predicciones en alta volatilidad simplemente no son de fiar bajo esa distribución desplazada. La respuesta fue deliberadamente sencilla: un filtro que solo deja operar cuando la volatilidad realizada a 5 segundos no supera el umbral 0,6657, que es el percentil 80 de la volatilidad de entrenamiento —es decir, un umbral fijado con los datos viejos, no ajustado a los nuevos.

Periodo	Fracción alta volatilidad	Efecto del filtro
Entrenamiento (11–20 may)	22 %	marginal
Validación (21–22 may)	21 %	marginal
Test (23–25 may)	18 %	marginal
Fuera de muestra (6–10 jun)	58 %	determinante (+0,73)

Tabla 3: El cambio de régimen entre entrenamiento y conjunto fuera de muestra justifica el filtro de volatilidad.

6. Resultados

6.1. Validación fuera de muestra

Con el candidato ya congelado y el filtro de volatilidad puesto, lo soltamos sobre los cinco días del conjunto fuera de muestra (6–10 de junio de 2026). La Tabla 4 recoge lo que salió. El intervalo de confianza es bootstrap con 5 000 remuestreos y semilla fija, y la probabilidad de neto positivo se estima sobre esa misma distribución de remuestreo.

Métrica	Valor
Operaciones (n)	318
Días de soporte	5 (5/5 positivos)
Neto medio (coste 0,5)	+1,069 ticks
Neto medio (coste 0,25)	+0,819 ticks
Intervalo de confianza 90 %	[+0,472, +1,654]
$P(\text{neto} > 0)$ (bootstrap)	99,8 %
Drawdown máximo	88,1 ticks
Neto medio sin filtro de volatilidad	+0,349 ticks

Tabla 4: Validación fuera de muestra del candidato congelado (ticks netos por operación). El filtro de volatilidad aporta +0,72 ticks, la mayor parte de la ventaja, precisamente por el cambio de régimen observado.

Conviene no perder de vista una cosa: el soporte son cinco días. Es poco, y lo decimos abierto junto a cada cifra en lugar de esconderlo. El filtro de volatilidad se lleva la mayor parte del mérito, y no por casualidad, sino porque el régimen de estos días difiere del de entrenamiento: sin el filtro, el neto medio se queda en +0,349 ticks.

6.2. Experimentos de adaptación al régimen

Ante el cambio de régimen, la tentación obvia era adaptar el modelo, y lo intentamos por varias vías: reentrenamiento incremental con ventana móvil, alineación de correlaciones CORAL [13], un filtro conformal, un *ensemble* por *bootstrap*, un filtro de volatilidad adaptativo, características normalizadas por volatilidad y *distillation* de una red convolucional temporal. Ninguna le ganó de forma robusta al modelo congelado con su filtro simple. Damos cuenta de este resultado negativo porque dice algo: con tan pocos días, un modelo congelado más un filtro sencillo le gana a la adaptación de dominio sofisticada, que necesita más datos de los que tenemos para estimarse sin hacerse ilusiones.

6.3. Seguimiento *paper-shadow*

La pieza de evaluación final es un registro *paper-shadow*: contabilidad operación a operación, resumen diario y reglas de promoción cuantitativas. Las reglas son adrede conservadoras y exigen soporte temporal, no solo neto positivo: para promocionar a candidato de seguimiento hacen falta un neto por encima de 0,5 ticks, al menos 200 operaciones y al menos 8 días. Con cinco, el candidato se queda en *prometedor pero con datos insuficientes* —le falta soporte temporal, no señal. Forzar el ascenso aquí sería, sencillamente, hacer trampa al método.

7. Discusión y limitaciones

La limitación principal es evidente y la repetimos sin maquillarla: cinco días de soporte fuera de muestra. El intervalo de confianza es estrecho y los cinco días dan positivo, sí, pero cinco días no son una validación definitiva. Hay más letra pequeña. El resultado es específico de la ventana *prestart* y del régimen capturado, y un cambio de régimen lo bastante brusco podría dejar inservible el filtro de volatilidad —el propio salto que vimos entre entrenamiento y fuera de muestra es la prueba de que eso pasa. Y la validación es *offline*, con un modelo de ejecución conservador pero simplificado; queda pendiente una simulación de ejecución de mayor fidelidad, con rellenos parciales y dinámica de cola.

Hay además un detalle que nos llamó la atención: dentro del rango que habilita el filtro, la relación entre volatilidad y beneficio dibuja una U, y es la volatilidad muy baja la que concentra casi todo el beneficio. Como descansa sobre los mismos cinco días, lo dejamos anotado como hipótesis a comprobar, no como resultado.

8. Consideraciones éticas

Un sistema que toma decisiones económicas solo, y que además es frágil al sobreajuste cuando hay pocos datos, obliga a ir con cuidado. Por eso el trabajo se para a propósito en la fase de *paper-shadow* y no llega a operar con capital real. Trabajamos con un marco de confidencialidad, integridad y disponibilidad, y tratamos la integridad de los datos como algo tan crítico como el propio modelo: una referencia externa que llega tarde o mal puede fabricar una señal falsa y, detrás de ella, una pérdida real. De ahí las decisiones concretas: entrenar solo con sesiones aceptadas, separar los datos crudos de las características y las etiquetas, mantener trazabilidad de cada artefacto y fijar umbrales que cortan la operativa en cuanto los datos se quedan obsoletos. No se usa ningún dato personal: solo datos públicos de mercado.

9. Conclusiones

Lo que queda al final es, sobre todo, una metodología: un sistema de captura de datos, una validación temporal y un control de costes para predecir el corto plazo en los mercados de Bitcoin de Polymarket. Y una observación que lo atraviesa todo: los clasificadores direccionales, por mucho que acierten cerca del 90 %, dejan de ser rentables en cuanto la latencia de entrada es realista; lo que decide es el coste y la ejecución, no el acierto. El aprendizaje profundo sobre secuencias y libro de órdenes aporta representación, pero no una política rentable y estable con unas dos semanas de datos. Quien sí sobrevive es un especialista tabular congelado con filtro de volatilidad, que aguanta una validación fuera de muestra de cinco días (+1,069 ticks netos por operación, cinco de cinco días positivos) y se queda en seguimiento *paper-shadow* con reglas de promoción cuantitativas. No afirmamos rentabilidad real, y esa contención es parte del mensaje: lo que se entrega es la metodología, la evidencia obtenida con un protocolo honesto y los límites dibujados sin disimulo. El camino a partir de aquí está claro: ampliar el soporte temporal fuera de muestra, comprobar si el patrón en U de la volatilidad se sostiene y volver a probar las arquitecturas profundas con muchos más días —pero solo si consiguen ganarle al especialista tabular bajo el mismo protocolo. El código, los registros de resultados y los artefactos para reproducir el trabajo se publican junto a este artículo, con las prácticas de trazabilidad que ayudan a contener la deuda técnica propia de estos sistemas [15].

Referencias

- [1] Polymarket Documentation. *Prices & Orderbooks*. <https://docs.polymarket.com/polymarket-learn/trading/using-the-orderbook>
- [2] Polymarket Documentation. *Order Lifecycle*. <https://docs.polymarket.com/concepts/order-lifecycle>
- [3] J. Wolfers and E. Zitzewitz. *Prediction Markets*. Journal of Economic Perspectives, 18(2), 107–126, 2004.
- [4] Z. Zhang, S. Zohren and S. Roberts. *DeepLOB: Deep Convolutional Neural Networks for Limit Order Books*. IEEE Transactions on Signal Processing, 67(11), 3001–3012, 2019.
- [5] A. Ntakaris, M. Magris, J. Kannianen, M. Gabbouj and A. Iosifidis. *Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods*. Journal of Forecasting, 37(8), 852–866, 2018.
- [6] A. Tsantekidis, N. Passalis, A. Tefas, J. Kannianen, M. Gabbouj and A. Iosifidis. *Using deep learning to detect price change indications in financial markets*. European Signal Processing Conference (EUSIPCO), 2017.

- [7] J. Sirignano and R. Cont. *Universal features of price formation in financial markets: perspectives from deep learning*. Quantitative Finance, 19(9), 1449–1459, 2019.
- [8] R. Jha et al. *Deep Learning for Digital Asset Limit Order Books*. arXiv:2010.01241, 2020.
- [9] M. Kearns and Y. Nevmyvaka. *Machine Learning for Market Microstructure and High Frequency Trading*. In High Frequency Trading: New Realities for Traders, Markets and Regulators, 2013.
- [10] S. Gu, B. Kelly and D. Xiu. *Empirical Asset Pricing via Machine Learning*. The Review of Financial Studies, 33(5), 2223–2273, 2020.
- [11] M. López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.
- [12] D. H. Bailey and M. López de Prado. *The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality*. The Journal of Portfolio Management, 40(5), 94–107, 2014.
- [13] B. Sun, J. Feng and K. Saenko. *Return of Frustratingly Easy Domain Adaptation*. AAAI Conference on Artificial Intelligence, 2016.
- [14] B. Lim, S. O. Arik, N. Loeff and T. Pfister. *Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting*. International Journal of Forecasting, 37(4), 1748–1764, 2021.
- [15] D. Sculley et al. *Hidden Technical Debt in Machine Learning Systems*. Advances in Neural Information Processing Systems (NeurIPS), 2015.